

VETS: Verification of LLM-Generated Smart-Home Automation via Executable Timeline Specifications

Anonymous Authors

Abstract—Large language models (LLMs) enable users to create smart-home automation code through natural-language requests. However, implicit temporal semantics and dynamic behavior make it difficult for humans or LLM judges to assess behavioral correctness from source code alone. We present VETS, a framework for generating automation code from natural-language requests and verifying its behavior before deployment. VETS uses a user-confirmed intermediate representation (IR), Timeline IR, to guide code generation through its control-flow structure and timing conditions, while its execution semantics provide the behavioral reference for verification. Behavioral Explorer formally verifies behavioral equivalence between the confirmed Timeline IR and generated code by jointly exploring their executions under shared inputs. Evaluated on 200 IR–code pairs, Explorer yielded definitive verdicts for 184 verifiable pairs—achieving 100% accuracy against independent reference checks. By employing input grouping and timer relations to mitigate state-space explosion, Explorer rapidly verified 28 out of 30 parameterized automations, completing 24 within 5 seconds.

Index Terms—IoT automation, LLM code generation, reactive-temporal code, deterministic verification, on-device, trace equivalence

I. INTRODUCTION

Internet of Things (IoT) automation is moving toward code generation with large language models (LLMs) [1], [2]. A user states the desired behavior in natural language, and an LLM turns it into a reactive rule the platform can execute. Yet the final step of this workflow, deciding *whether the generated code is correct to deploy*, has no deterministic gate. The missing piece is a reference to check the code against; the natural-language request contains the intent, but not in a form that can serve as a rigorous baseline for automated testing.

Why this is hard: The core challenge stems from the absence of a verifiable reference baseline. The target automations are inherently *reactive-temporal*. They are *reactive* because their execution is driven by asynchronous events and conditional triggers, and *temporal* because their behavior unfolds over time through schedules, periodic executions, and sustained state durations (e.g. “if it persists for more than N minutes”). Driven by timers, counters, and multi-step state, the correct behavior of an automation must be evaluated across an entire timeline rather than a single discrete instant. Furthermore, in our setting, these temporal behaviors are deployed as imperative code implemented through variable updates and conditionals. Validating their correctness presents two key challenges. First, no request-specific test oracle is provided in advance. Unlike text-to-SQL (Structured Query Language) or traditional programming benchmarks that supply gold denotations or input-output test suites [3], [4], a reactive

automation’s output is a future sequence of environment interactions. Therefore, a behavioral reference for verification must be constructed from the request. Second, syntactic variation complicates evaluation. The same user intent can be expressed through many syntactically different yet behaviorally equivalent implementations. Therefore, checking a generated program by matching it against any single reference text is neither sound nor complete. Determining correctness fundamentally requires reasoning about the program’s semantic behavior over time, rather than analyzing its static syntax or a single snapshot of its output.

Running example: Suppose a user requests “whenever the temperature rises above 25°C, turn on the air conditioner.” We read “rises above” as an edge-triggered intent: the air conditioner should be switched on once per upward crossing, not continuously while it stays hot. Figure 1 shows three lowerings, sketched in JOI [5], the automation domain-specific language (DSL) of our target platform. All three implementations execute with a one-second tick period. Implementation (a) detects the crossing by comparing previous and current readings, while (b) uses a flag that is set on firing and reset below the threshold. They share almost nothing syntactically yet behave identically. Implementation (c) textually mirrors the request, but under periodic execution it falsely re-fires on every tick while the temperature stays above the threshold. Because repeated On() commands leave the air conditioner in the same visible state, the user may see nothing wrong. This is a crucial failure mode our system targets: **deployed code that silently diverges from the user’s intended behavioral specification**.

Why existing checks fall short. Prior verification frameworks of IoT automations target fixed, predefined criteria, such as safety properties, inter-rule conflicts, or security policies [6]–[8]. These checks establish whether an automation satisfies the selected properties, but do not by themselves establish whether it implements the particular behavior requested by the user. Another LLM could judge whether the code matches the request, but in our experiments, verdicts changed even under behavior-preserving temporal rewrites (§3). Therefore, a verification reference specifying the requested behavior and a method for consistently checking the code against it are needed.

VETS. To address these challenges, we present VETS. Its key idea is to *derive the missing reference specification* at authoring time and subsequently utilize it as a deterministic verification baseline. VETS represents the requested automation in an intermediate representation (IR) called Timeline

```

1 prev := 0 // persistent initialization
2 current = (#TemperatureSensor).temperature
3 if (prev <= 25 and current > 25) {
4   (#AC).on()
5 }
6 prev = current

```

(a) Previous/current comparison

```

1 triggered := false // persistent initialization
2 temp = (#TemperatureSensor).temperature
3 if (temp > 25) {
4   if (!triggered) {
5     (#AC).on()
6     triggered = true
7   }
8 } else {
9   triggered = false
10 }

```

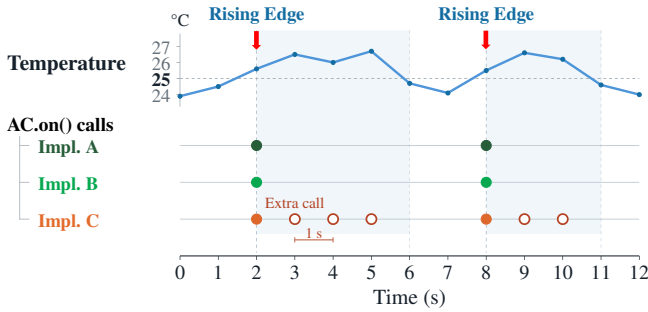
(b) Triggered flag

```

1 wait until ((#TemperatureSensor).temperature > 25)
2 (#AC).on()

```

(c) Re-fires while the level holds



(d) Timed action traces

Fig. 1. Three lowering sketches of one intent and their timed action traces. (a) and (b) produce identical traces; (c) fires on every tick while the threshold is exceeded. Platform boilerplate is omitted.

IR. This representation specifies the behavioral structure for the LLM to implement in code and provides an executable reference for checking the generated JOI code.

Timeline IR makes an automation’s temporal control flow explicit by specifying execution order and timing conditions (§V). It can thus clearly express automations that wait for an event, perform an action, and repeat with a delay until a termination condition is met. Given an initial state and a timed input sequence, which specifies input values and when they change, its execution semantics determine the expected timed action trace, consisting of device commands and their occurrence times.

Behavioral Explorer checks whether the IR and generated JOI code produce the same timed action trace for every allowed initial state and timed input sequence (§6). It certifies equivalence only when checking completes within the specified input ranges and execution rules and provides an executable counterexample for confirmed divergence, meaning a difference in actions. Unsupported or unfinished checks remain uncertified.

Scope and assumptions. During authoring, the user con-

firms the IR as the specification, and we assume that it captures the intended automation. We do not evaluate users’ ability to identify IR errors. VETS checks for state, timer, and boundary errors introduced during subsequent code generation, and its guarantee is limited to conformance of the generated code to the confirmed IR.

Local generation and verification. We assume a setting in which automation requests and device information are not disclosed to external services during generation or verification. To meet this requirement, VETS uses a local language model to generate both Timeline IR and JOI code and performs behavioral verification locally without additional LLM calls.

Contributions. Our contributions are:

- **C1. Timeline IR:** We introduce an executable intermediate representation that specifies an automation’s execution order and timing conditions. Its execution semantics define the expected behavior for a given initial state and timed input sequence, providing a behavioral reference for verifying generated JOI code.
- **C2. Behavioral Explorer:** We propose a deterministic checker that jointly explores Timeline IR and generated JOI code under the same timed input sequences and compares their timed action traces. It reports executable counterexamples for confirmed divergence and certifies equivalence only when exploration completes within the specified input ranges and execution rules.
- **C3. Implementation and evaluation:** We implement and evaluate VETS. Our evaluation examines Timeline IR’s expressiveness, the reliability of verification verdicts, the code generation and repair workflow, and verification cost and scalability.

II. RELATED WORK AND POSITIONING

Verifying automations generated from natural language requires checking not only whether selected properties hold but also whether the code behaves as the user requested. However, the natural-language request itself is not an executable reference for comparison. We therefore compare prior work by the artifact analyzed, checking reference, and method (Table I).

When a verification reference already exists. A rich body of work verifies trigger-action programming (TAP) rules with formal techniques. AutoTap [6] synthesizes or repairs TAP rules so that they satisfy user-specified safety properties expressed in linear temporal logic (LTL). TAPInspector [9] model-checks timing-aware rule sets for safety and liveness violations, and TAPFixer [13] repairs such violations. Related analyses uncover inter-rule and security vulnerabilities [14], [15], while HAWatcher [16] monitors deployed automations against mined invariants. These systems check properties of automation behavior, including temporal behavior and rule interactions.

Executable references are also available for some code-generation tasks. GPIOt [1] evaluates generated signal-processing and machine-learning algorithm code with execution tests, while text-to-SQL evaluates generated queries against the execution results of gold queries [3].

TABLE I

REPRESENTATIVE SYSTEMS BY ARTIFACT, CHECKING REFERENCE, AND METHOD. ENTRIES SUMMARIZE THE INDICATED CHECKS, NOT EVERY SYSTEM COMPONENT OR EVALUATION METRIC. TAP: TRIGGER-ACTION PROGRAMMING; LTL: LINEAR TEMPORAL LOGIC; NLI: NATURAL LANGUAGE INFERENCE; SMT: SATISFIABILITY MODULO THEORIES.

System	Artifact analyzed	Checking reference	Method
AutoTap [6]	TAP programs	User-specified LTL safety properties	Automata-based synthesis and repair
TAPInspector [9]	Timing-aware TAP rules	Safety and liveness properties	Model checking (NuSMV)
GPIOt [1]	Generated IoT algorithm code	Manually authored test cases	Execution tests (offline evaluation)
AutoIoT [10]	Models of generated TAP rules	Four conflict definitions	State-space search in Maude
ChatIoT [11]	Generated TAP representation	User request, context, and format requirements	LLM Evaluator
AwareAuto [2]	Reactive-temporal rules and grounded JSON	User intent and device interfaces	User revision and deployment checks
LACE [12]	Generated access-control policies	User request and policy-conflict definitions	NLI for meaning; SMT for conflicts
VETS	Reactive-temporal JOI code	User’s intended behavior specified in Timeline IR	Timed action-trace equivalence checking

LLM-generated reactive automations. Systems that generate automations from natural-language requests use different criteria and procedures to check their outputs. AutoIoT [10] checks generated rules against four inter-rule conflict types using Maude rewriting logic. ChatIoT [11] translates natural language into Home Assistant automations and uses an LLM Evaluator to assess format compliance and request satisfaction.

AwareAuto [2] constructs automation rules with event/state modes and timing conditions and supports user review, revision, and grounding in device interfaces. VETS differs in checking whether the state updates and action timing of JOI code, which implements such temporal behavior through variables and conditionals, preserve the behavior specified by the confirmed Timeline IR.

Outside automation, LACE [12] back-translates generated access-control policies into natural language and uses a natural language inference (NLI) model to judge semantic equivalence with the original request, while separately checking policy conflicts with a satisfiability modulo theories (SMT) solver.

We ask whether specified automation behavior is preserved in separately generated reactive-temporal code. VETS uses user-confirmed Timeline IR as a common reference for code generation and verification and checks whether the generated code’s timed action trace matches this reference.

III. PROBLEM AND MOTIVATION

Together, §1 and §2 motivate a deployment gate that checks whether generated code preserves the user’s intended behavioral specification.

Two key requirements for a deployment gate.

- **R1 (Reference).** In our authoring setting, no request-specific verification reference is supplied in advance. Algorithmic code evaluations with input-output test cases, such as GPIOt (§2), can use those cases to check generated code. Here, the question is whether the requested device actions occur correctly over time, but the request itself is not an executable specification. A verification reference that defines the expected behavior must therefore be constructed from the request.

- **R2 (Stability).** Programs with the same behavior must receive the same deploy/reject verdict against the same specification, and repeated evaluations must agree. A checker whose verdict depends on code presentation despite unchanged behavior is unsuitable as the sole deployment gate.

Candidate checking approaches. The most straightforward verification method is to match the generated code directly against a reference text. However, as the temperature example in §1 illustrates, identical behavior can be realized through syntactically disparate idioms, while a buggy implementation can closely resemble the request. One could instead ask the generating model to inspect and repair its code, but SimuHome [17] reports limited recovery from scheduling errors through self-correction. Showing the code to the user also requires inspection of its implementation details; TAP-Debug [18] found that non-experts frequently misread even the basic IF (event) and WHILE (state) conditions of raw rules. These limitations motivate a behavioral check that does not rely solely on textual similarity, model self-assessment, or users’ inspection of code.

An LLM judge [19]. Another approach is to hand the request and the code to a separate LLM to verify whether they align. This approach appears plausible at first glance and is used in practice (ChatIoT’s Evaluator [11], §2). To evaluate whether the judges satisfy R2, we submitted each natural-language request–program pair, using JOI programs (Section VII-C), three times to Qwen3.5-9B [20], [21], GPT-5.4-mini [22], and Claude-Sonnet-5 [23]. Each call also included language documentation, but the judge did not receive Timeline IR. Qwen used temperature 0 and seed 0, and GPT used a fixed seed of 42. We then selected the 52 originals accepted by each judge through majority vote and evaluated 168 variants with different code presentations whose behavior preservation was checked by Behavioral Explorer (§6). The rewrite rejection rate is the proportion of these variants that a judge marked as not correctly implementing the user’s request.

As Table II shows, GPT had a verdict disagreement rate of 16.6% across three identical submissions despite its fixed seed, while Claude had a rate of 9.7%. Thus, lower rejection rates on the variants in this experiment did not ensure repeatable ver-

TABLE II
VERDICT DISAGREEMENT ON UNCHANGED CODE: THE PERCENTAGE OF
217 PROGRAMS WHOSE THREE EVALUATIONS DID NOT ALL AGREE.

	Qwen3.5-9B	GPT-5.4-mini	Claude-Sonnet-5
Disagreement (%)	0.0	16.6	9.7

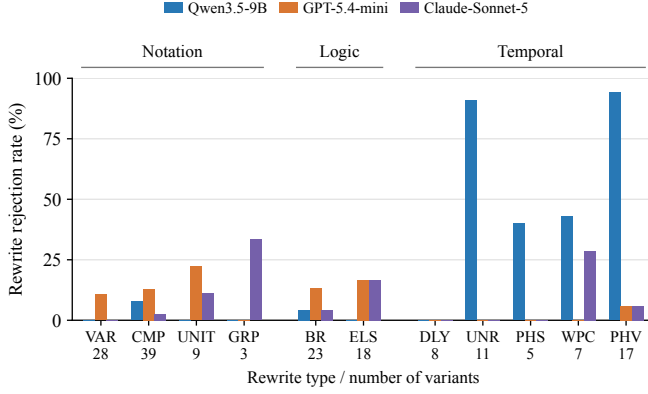


Fig. 2. Rejection of behavior-preserving rewrites of the 52 commonly accepted originals. Counts appear below each type. VAR: variable renaming; CMP: comparison reversal; UNIT: time-unit conversion; GRP: group-condition respelling; BR: branch swap; ELS: else splitting; DLY: delay splitting; UNR: loop unrolling; PHS: phase-to-flag conversion; WPC: wait precheck; PHV: period halving with alternate execution.

dicts on unchanged code. By contrast, under the settings above, Qwen had a verdict disagreement rate of 0.0% on unchanged code. Yet it had the highest overall rejection rate among the three judges on variants that preserved behavior. Qwen largely maintained its acceptance verdicts under notation and logic changes, but frequently rejected behaviorally equivalent code when its temporal structure changed. The rejection rate for temporal variants reached 64.6%. The individual transformations in Figure 2 show rejection rates of 90.9% for loop-unrolling variants (UNR) and 94.1% for variants that halved the execution period while running the body on alternate iterations (PHV).

Across all variants, the cloud judges GPT and Claude had lower rejection rates, 10.1% and 6.0%, respectively, than Qwen at 20.8%. These results concern verdict consistency, not bug-detection accuracy. The rewrite results are conditional on the common accepted set. Because GPT and Claude already varied on unchanged code, and the table and figure use different populations and measures, their rates do not isolate an additional effect of rewriting. Under the tested conditions, the observations undermine reliance on these judges alone for a stable deployment decision; they do not rule out every possible LLM-based checker.

Design direction. The requirements and observations point to an explicit behavioral reference and a separate execution-based check. By R1, the reference must be constructed during authoring, and by R2, the check must compare behavior rather than depend on code presentation. VETS uses Timeline IR (§5) to make the automation’s temporal control flow explicit and

Behavioral Explorer (§6) to check the generated code against it. As stated in §1, the guarantee assumes that the confirmed IR captures the intended automation.

IV. SYSTEM OVERVIEW

VETS implements the aforementioned design as a two-phase pipeline. In the authoring phase, the system constructs a Timeline IR from the user’s request and converts it to executable JOI code after the user reviews and confirms its behavior. In the verification phase, Behavioral Explorer checks the generated code against the confirmed IR before deployment.

Execution target. VETS generates and verifies JOI code for execution on the edge hub of a commercial IoT platform. A JOI automation consists of a start schedule (`cron`), a re-execution interval (`period`), and a script body (`code`). Temporal behavior such as edge detection, sustained conditions, and counting must be implemented with persistent variables and ordinary conditionals in the repeatedly executed script.

Phase 1: Authoring. VETS first analyzes the user’s natural-language request. This analysis identifies the temporal and behavioral slots required for the automation, such as the start time, repetition period, trigger or condition, delay or sustain duration, and target actions. It also establishes device bindings by matching devices mentioned in the request to the connected device list. Based on this analysis, a local LLM generates a Timeline IR that specifies the automation’s temporal control flow. Section V describes the structure and presentation of Timeline IR.

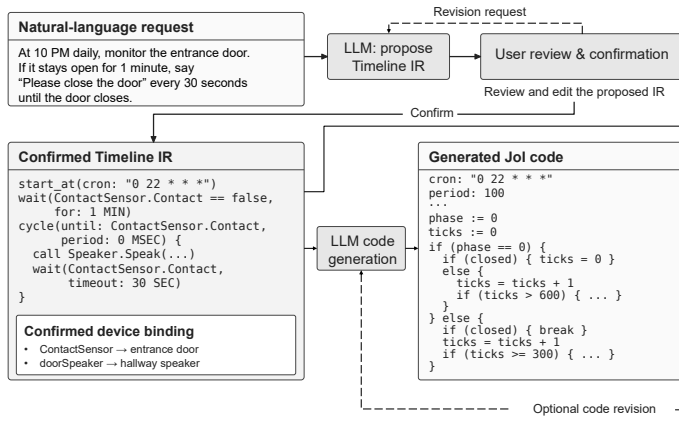
The user reviews the IR’s behavior and device bindings and may request revisions before confirmation. After confirmation, the IR is fixed as the behavioral reference, and the local LLM generates JOI code from the confirmed IR and bindings.

Phase 2: Verification. Behavioral Explorer parses the generated JOI code and configures the device binding, start time, input ranges, and execution rules. Starting from the allowed initial states, it jointly explores the IR and code under shared inputs and logical time, comparing their timed action traces.

Only a completed exploration establishing matching timed action traces for every allowed initial state and timed input sequence authorizes deployment. A divergence is reported only after replay confirms the mismatch; unsupported operations, execution errors, or exhausted checking budgets leave the pair uncertified and do not authorize deployment. Verification runs locally without LLM calls. Its guarantee concerns conformance to the confirmed IR within the specified input ranges and execution rules. §6 describes the joint exploration procedure and verdict criteria in detail.

Counterexample feedback. A counterexample records the initial environment, timed input sequence, and expected and actual actions. It can be returned to the LLM to guide revision of the JOI code. The confirmed IR and binding remain unchanged, and the revised code must be checked again before deployment.

Authoring Phase



Verification Phase

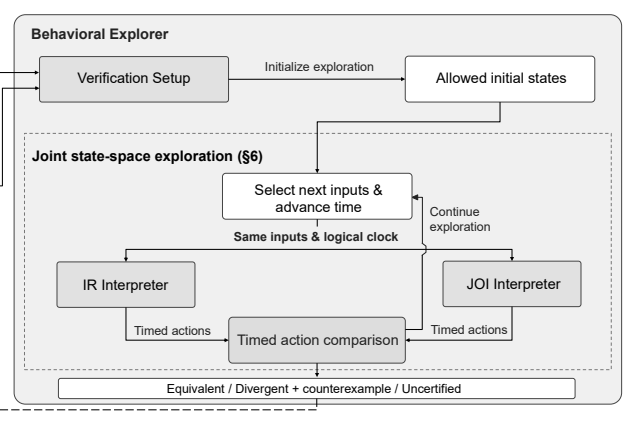


Fig. 3. VETS system overview. Authoring generates JOI code from a user-confirmed Timeline IR, and verification compares the generated code's behavior with the confirmed IR. Dashed arrows indicate user-requested IR revision and optional counterexample-guided code revision.

```

01 {
02   "timeline": [
03     {"op": "start_at", "anchor": "now"},
04     {"op": "wait", "cond": "ContactSensor.Contact == false",
05      "edge": "rising"},
06     {"op": "call", "target": "Switch.On", "args": {}},
07     {"op": "cycle", "period": "0 SEC",
08      "until": "ContactSensor.Contact == true",
09      "body": [
10       {"op": "call", "target": "Speaker.Speak",
11        "args": {"Text": "Please close the door."}},
12       {"op": "wait", "cond": "ContactSensor.Contact == true",
13        "timeout": "30 SEC"}
14     ]},
15     {"op": "call", "target": "Switch.Off", "args": {}}
16   ]
17 }

```

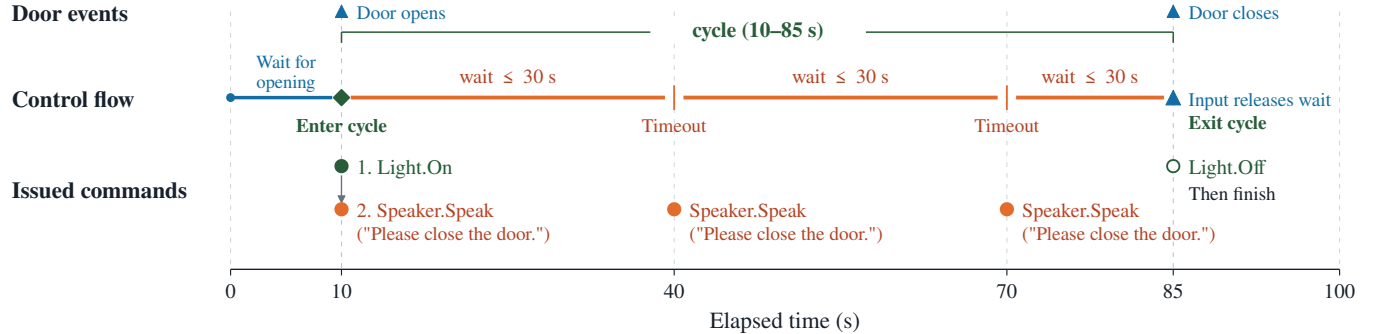
(a) Timeline IR

```

01 Start now.
02 Wait until the door-open condition becomes true.
03 Turn on the entrance light.
04 Repeat until the entrance door is closed:
05   Run Speaker.Speak(Text=Please close the door.).
06   Wait until the entrance door is closed
07   or 30 seconds elapse, whichever is first.
08   If closed, stop the current repeat loop.
09   Otherwise, repeat with no additional wait.
10 After the loop:
11 Turn off the entrance light.
12 Finish.

```

(b) Plain-language rendering



(c) Execution under one timed input sequence

Fig. 4. Timeline IR example: (a) structured representation, (b) plain-language rendering, and (c) execution under a timed input sequence.

V. TIMELINE IR

Timeline IR makes an automation's temporal control flow explicit along a time axis. Schedules, waits, and delays determine when execution proceeds; sequences, branches, and cycles determine how it proceeds. Once confirmed by the user, the IR guides code generation and provides an executable behavioral reference for checking the generated JOI code.

Operators and structure. Structurally, an IR is a tree of JSON steps, each identified by an `op` field. A timeline begins with `start_at`, which sets the start time, and executes

its steps in order. The `if` branches and `cycle` bodies contain further sequences. The primitive operators also include `wait` for a condition, an observed edge, or a sustained condition; `delay` for a duration; `read` for saving a value; and `call` for invoking a device service. A sustained wait resets its timer if the condition becomes false. A cycle waits for its `period` after each completed iteration and can use a count or an `until` condition to stop; `break` exits the enclosing cycle. One timeline has no parallel branches. The confirmed binding identifies the concrete devices referenced by the IR.

The example in Figure 4 starts with the door closed; `Contact` is true when closed, and `Switch` is bound to the entrance light. The IR in Figure 4(a) waits for the door-opening event, turns on the light, and enters a cycle. The body issues the reminder “Please close the door” and waits until the door closes or 30 seconds elapse. If the door remains open when the wait times out, the next iteration issues another reminder. When the door closes, the wait ends and the `until` condition terminates the cycle. The `call` following the `cycle` (line 15) then turns off the light. The cycle is thus an intermediate part of the control flow, with execution continuing to the subsequent step after it ends. Figure 4(b) expresses this control flow in plain language.

Figure 4(c) shows how this control flow progresses along the time axis as the door state changes. When the door opens at second 10, the IR turns on the light and issues the first reminder. The door remains open, so reminders follow at seconds 40 and 70. When the door closes at second 85, execution exits the cycle and turns off the light without waiting for the timeout at second 100. Elapsed time advances execution to the next iteration, whereas door closure advances it to the step after the cycle.

Executable reference. The IR interpreter retains the execution position, stored values, and timers. At each relevant time, it executes steps until waiting or termination, recording issued commands and their times as a timed action trace. When an input update coincides with a timer deadline, it applies the new input first. With the IR, binding, and execution rules fixed, the same initial state and timed input sequence determine the same commands, in the same order and at the same times. This input-determinism assumes error-free execution with finitely many steps in each finite time interval. Section VI uses this expected trace to check the generated code across allowed inputs.

Presenting the IR. Users can inspect the JSON directly or review a plain-language rendering such as Figure 4(b). A time-axis view such as Figure 4(c) illustrates behavior under selected inputs. This structure could be extended to let users change input values or their timing and inspect the resulting behavior, or to generate explanations with an LLM. These extensions and their effects on user understanding are not evaluated here. Across presentation methods, the confirmed IR remains the specification, and we assume that it captures the user’s intended behavior.

VI. CODE GENERATION AND BEHAVIORAL VALIDATION

As shown in Figure 3, the LLM generates JOI code using the execution order and timing conditions specified in the confirmed Timeline IR as guidance and applying the device binding. Behavioral Explorer checks whether the IR and generated code produce the same timed action trace for every allowed initial state and timed input sequence within the specified input ranges and execution rules. This check of each generated artifact follows the translation-validation approach [24].

A. Comparing Behavior under Shared Inputs

Checking compares the two programs’ behavior under the same inputs and tracks how values retained from one reaction affect later behavior. Explorer uses a semantic interpreter for each program to compute its next actions and state. A state comprises the execution position, stored values, timers, and other information that determines subsequent behavior. Both interpreters receive the same external input values and logical time, while maintaining separate states according to their own execution rules.

Each interpreter records the device commands it issues, together with their times, to form a timed action trace. Explorer compares the target, method, arguments, number, and order of commands at each time. Internal assignments and control steps need not match. In the temperature example from §1, an extra `On()` command while the temperature remains high exposes a mismatch even though the air conditioner stays on in both executions. A missing or delayed command also produces a mismatch.

To make this comparison meaningful, Explorer sets the device binding, start time, allowed initial states, and input ranges before checking. Inputs such as sensor values and device states may change at regular update times and remain constant between updates. Explorer considers input updates, timer deadlines, and periodic code executions that can affect either program. When it can establish that no intervening change affects actions or subsequent execution, it skips intermediate time points and advances to the next relevant event. This advances logical time without waiting in real time, while preserving elapsed time and action timestamps.

B. Exploring Possible Executions

Consider a simple sustained-condition automation: send a notification if a door remains open for 30 seconds. Behavior depends on whether the door closes before, at, or after the deadline. If it closes before the deadline and then reopens, the wait must restart.

Starting from the allowed initial states, Explorer considers each possible next input, advances both executions, and compares their commands. If the resulting state matches one already checked, Explorer reuses the subsequent checks; otherwise, it adds the new state to the remaining work [25]. Returning both programs to a previously checked waiting state after the door closes is one example of reuse. Repeating this process covers sequences in which the door closes and reopens multiple times.

Enumerating every input value and every repetition can be expensive. Explorer reduces this work where it can preserve the behavior being compared. When input values are used only to evaluate conditions, Explorer groups values that produce the same result for every condition in both programs. For example, if we check whether temperature exceeds 22 degrees, Explorer needs to distinguish two groups of values only, values at or below 22, and above 22, instead of taking each value as separate. On the other hand, if an input value is used as an action argument or in another computation, Explorer checks

differences in the actual values because they can change the action arguments or subsequent behavior.

For time, Explorer can also reduce the cost of checking many states that differ only in elapsed time. Where supported, it represents multiple time values together through relations and checks successor behavior over the entire represented range. This representation uses difference-bound constraints from timed verification [26]. Returning to the door example, it can jointly represent waiting states before the 30-second deadline while preserving resets on door closure and the exact deadline.

This requires relating the two programs’ different representations of time. The IR interpreter uses timers to manage waits. JOI code can implement the same wait by counting periodic executions or by saving a start time and computing the difference from the current time. For code using a counter, Explorer checks its initialization, increments, and resets to maintain the relation between the counter and elapsed time in the IR. It checks commands and successor states for each possible input over the states represented by these relations. If the represented range is enlarged to include a newly reached state, Explorer rechecks successor behavior over that enlarged range [27]. Thus, it explores the possible system states incrementally for both IR and JOI interpreters. In case the number of states is not finite, Explorer’s verdict is *“unsupported”*. If the number of system states increases explosively, *“timeout”* may occur within the specified deadline of Explorer execution.

C. Verdicts and Feedback

Explorer reports a divergence only after replaying a timed input sequence from initialization and confirming a difference in the timed action traces. It returns the initial environment, that input sequence, and expected and actual actions. The LLM can use this evidence to revise the code; Explorer checks the revision against the same IR and binding.

Equivalence requires a completed check. All possible terminating paths must have matching timed action traces. For automations that repeat without terminating, every possible continuation must lead to states already covered by the check, with matching actions and timing. When states are grouped, these conditions must hold for all represented states. Unsupported operations, execution errors, or exhausted checking budgets leave the pair uncertified and do not authorize deployment.

VETS implements Explorer in Python and uses Z3 [28], a tool for reasoning about expressions and constraints, to compare arithmetic expressions. Checking uses semantic interpreters without LLM calls. The equivalence guarantee applies within the specified input ranges and execution rules, assuming correct parsing, interpretation, and Z3 results. Appendix A sketches the soundness argument under these assumptions. The guarantee does not establish that the confirmed IR captures the user’s intent.

VII. EVALUATION

Setup. All code evaluations use JOI and the execution model in Sections V and VI. Four sets of experiments were

conducted: E1 evaluates expression and reference execution, E2 checks verdict fidelity, E3 evaluates generated JOI code, and E4 measures completion and cost. Refusals, incomplete searches, and errors remain in each evaluation’s denominator. LLM generation is separate from behavioral checking.

Datasets. We use 382 author-written commands from a test set created for the JOI platform to evaluate generated-code validation and repair. The set varies automation type and structural complexity across 24 defined patterns, including conditional and event-triggered actions, sustained conditions, delayed sequences, schedules, and repetition. Its confirmed IRs contain two to eight operator nodes, with a maximum nesting depth of zero to three for branches and cycles. The evaluation datasets is available in our artifact [5].

We additionally collected 100 external automation requests to evaluate Timeline IR adequacy (E1). The following subsection describes their sources and behavioral characteristics.

A. Timeline IR Adequacy

E1 evaluates whether Timeline IR can express automation requirements drawn from outside our own JOI test set. We collected 100 requests from official Google Home and Home Assistant examples [29]–[32], published research tasks and examples [6], [9], [33]–[35], released AutoTap participant responses [36], and Home Assistant Community posts [37]. During collection, we sought complex requests combining timing requirements and multiple execution stages.

The collected requests were reviewed to identify reactive-temporal scope, semantic duplicates, and unresolved behavioral choices. Of the 100 requests, 92 were in scope; six were classified as ambiguous because their source descriptions are unclear with respect to execution time windows or conditions for acting or interrupting execution. The remaining two were excluded since they involved online-service integration, such as updating a Twitter profile picture when the Facebook profile picture changes. The evaluated requests range from basic trigger-action rules to compound conditions and multiple device actions, sustained conditions, delayed sequences, repetition, and history-dependent execution. They also include response timeouts, cancellation or restart during execution, restoration of prior state, and independently progressing flows. These features can occur together within one request. For example, a request to blink a light repeatedly while a garage door is open, stop when it closes, and restore the original light setting combines repetition, cancellation, and state restoration [38].

We encoded and tested all 92 requests. Their final IRs contain three to eight distinct operator types per request, including `start_at` and `call`, with maximum branch/cycle nesting depths ranging from zero to four. These statistics describe the encodings, including monitoring loops used to detect events. For each request, we recorded a concrete interpretation and expected actions and timestamps before encoding it. We replayed the input histories on the reference IR runner and compared actions and timestamps, treating simultaneous actions as an unordered group. For all 92 evaluated requests, denoted as E1 requests in the subsequent experiments, the final IR reproduced

TABLE III
AGREEMENT BETWEEN EXPLORER VERDICTS AND INDEPENDENT
REFERENCE CHECKS. EQUIVALENCE IS CHECKED ON TESTED INPUTS;
DIVERGENCE BY COUNTEREXAMPLE REPLAY. DASHES DENOTE NO
DEFINITIVE VERDICT.

Verdict	Pairs	Supported	Contradicted
Equivalent	64	64	0
Divergent	120	120	0
Unsupported	8	–	–
Timeout	8	–	–
Total	200	184	0

the expected actions and timestamps on every test input prepared for that request. This supports the IR’s ability to express the requests under their specified interpretations and execution conditions and reproduce the expected behavior on the tested inputs.

B. Verdict Reliability

E2 evaluates whether Explorer’s equivalence and divergence verdicts agree with independently implemented reference executions. We constructed 150 pairs IR–JOI pairs from 20 E1 requests and 130 variants that were created by modifying the JOI code generated from those 20 requests. The modifications span 12 families, including action omission, timing, guards, stored values, order, and repetition. In addition, we included 50 valid LLM-generated candidates sampled from the 382-request development set.

Verdict agreement. As displayed in Table III, all 184 verdicts issued by Explorer agreed with the reference checks. The 64 equivalent pairs matched in actions and timestamps on every tested input, and all 120 divergent pairs exhibited a behavioral difference when their counterexamples were replayed. Thus, no incorrect equivalence verdict was observed on the tested inputs, and every reported divergence was reproduced.

Decision coverage. The remaining 16 pairs received no definitive verdict. Timeout verdict was made for 8 pairs that have explosive number of states to explore within the 10 minute time bound. The other eight were unsupported because of limitations in accumulated-variable analysis. For example, seven pairs implement or modify a request that extends the termination time by two minutes whenever motion is detected, with no upper bound on the accumulated count. It incurs infinite number of value states to explore.

C. Code Generation Workflow

E3 evaluates a code generation workflow that starts from confirmed Timeline IR and combines code generation, behavioral validation, and counterexample-guided repair. Divergent candidates were revised once using counterexample feedback and then rechecked. For each of the 382 JOI commands, we supplied the confirmed IR and binding to Qwen3.5-9B [20], [21] to generate JOI code.

Explorer issued a semantic verdict for 377 candidates (Table IV), among which 68 divergences were confirmed by its concrete replay. The unsupported cases involved a binary

TABLE IV
OUTCOMES ON GENERATED JOI CANDIDATES.

Outcome	Candidates	Share
Equivalent	309	80.89%
Divergent (replay-confirmed)	68	17.80%
Unsupported	3	0.79%
Timeout	2	0.52%
Total	382	100.00%

return assignment, unbounded temperature arithmetic, and a large illuminance domain. Note that these exploratory results are relative to the confirmed IR and binding, not measurements of natural-language intent accuracy or physical-device behavior.

Counterexamples for repair. Using counterexample feedback, we asked the same model to revise each of the 68 divergent candidates once, keeping the IR unchanged. Rechecking certified 36 (52.9%) as equivalent; 26 remained divergent, and six attempts failed due to timeouts.

Verification cost. For all 382 initial candidates except the three rejected before checking, median verification time was 72 ms and the 95th percentile was 158 ms, including process startup and preparation.

D. Validation Time Complexity

E4 evaluates how checking time and completion change as the system complexity increases in terms of the number of states to explore. In particular, it examines the effect of checking multiple time states together, as described in Section VI. Using one program template, we varied the number of Boolean sensors (1–7), sequential wait–call stages (1–6), wait duration (100 ms–4 h), and repetitions (1–200). We constructed 30 equivalent IR–JOI pairs using settings that isolate each factor and settings that jointly increase sensors, stages, and repetitions. The combined settings fix the wait duration at 2 min and reach seven sensors, six stages, and 100 repetitions.

The baseline is **explicit-state exploration** [25], obtained by disabling Explorer’s analysis path that groups time states. Both methods use the same interpreters, skip intermediate time points that cannot affect execution, and advance to the next relevant event. Both also reuse identical states already visited. But the baseline checks different elapsed or remaining times as separate states even at the same execution stage. Thus, when a program must keep checking inputs during a wait, longer waits can require visits to many states that differ only in time values.

E4 compares the effect of this time-state reduction on checking completion and cost. Both methods require complete exploration to establish equivalence. Each program ran three times per method with a 120 s budget per run. On an eight-core host, Explorer ran serially and explicit-state exploration used four concurrent workers.

Explorer completed 28 of the 30 programs, compared with 12 for explicit-state exploration (Fig. 5). This experiment confirms the efficiency of grouping time states on the state

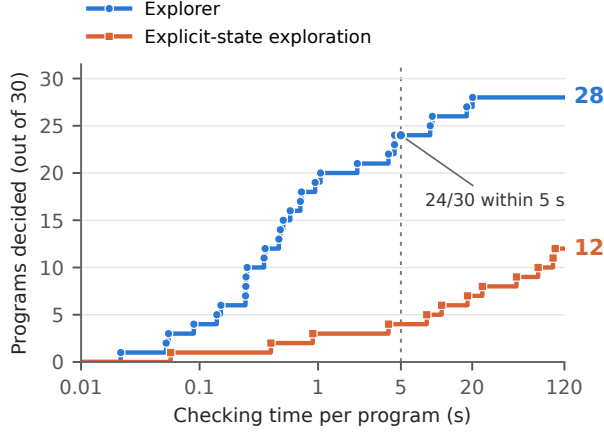


Fig. 5. Cumulative number of programs decided within each checking time (logarithmic time axis). A program counts as decided only if all three runs complete; its time is the slowest of the three.

exploration time complexity. Its two unfinished programs combined six sensors, six stages, and 50 repetitions, or seven sensors, six stages, and 100 repetitions, having explosively increasing number of states to explore. Nonetheless the proposed Behavior explorer is proven affordable to formally validate the generated JOI code against the associated Timeline IR for natural language requests that incurs a limited number of possible system states. Both reached the time limit.

VIII. LIMITATIONS AND FUTURE WORK

VETS checks generated code against user-confirmed Timeline IR. Whether the IR correctly captures the natural-language request remains a separate challenge. The execution order and timing conditions made explicit in the IR could support plain-language explanations or time-axis views for user review. Whether these representations help users understand the behavior and identify errors requires future user studies.

Explorer’s verdicts hold within the specified input ranges, execution rules, and device bindings, assuming correct parsing, interpretation, and constraint solving. Implementations outside the supported analysis, such as those with unbounded accumulated variables, and searches that cannot finish within the available resources receive no definitive verdict. The check compares device commands and their timestamps; it does not guarantee physical behavior under communication delays or device failures.

The evaluation is limited to JOI and the program structures examined. Applicability and checking cost on other platforms and automation structures require further evaluation. Future work will extend language and platform support and evaluate long-term operation in real environments.

IX. CONCLUSION

We presented VETS to generate smart-home automation code from natural-language requests and validate its behavior before deployment, using user-confirmed Timeline IR as a

common reference for code generation and validation. Timeline IR makes execution order and timing conditions explicit to support user review and code generation. Behavioral Explorer checks whether the IR and code produce the same actions at the same times under shared inputs. It provides concrete counterexamples when behavior diverges and does not certify code when checking remains incomplete.

We encoded all 92 in-scope external requests in Timeline IR and reproduced the expected behavior in request-specific tests. In the verdict reliability evaluation, all 184 verdicts issued by Explorer agreed with independent reference checks. In the cost evaluation, Explorer completed checks for 28 of 30 programs within a 120s budget. These results support checking for behavioral differences between a confirmed specification and generated code before deployment, while correctly interpreting natural-language intent remains a prerequisite.

APPENDIX A

EXPLORER SOUNDNESS: PROOF SKETCH

Fix the bindings, start time, input ranges, initial states, and execution rules of Section VI as M . Assume correct parsing, interpretation, and Z3 results, finite execution at each logical time, and no infinite resumptions within finite time.

Claim. An equivalence verdict implies equal timed action traces for every initial state and timed input sequence allowed by M .

Proof sketch. Let R include all IR–code state pairs represented by the completed check. Completion requires

$$\text{Init}_M \subseteq R, \quad \text{Post}_M(R) \subseteq R.$$

Here, Init_M contains all allowed initial pairs, and $\text{Post}_M(R)$ contains every allowed successor of R . Commands and times must match throughout R . Terminated programs issue no further commands.

Grouping inputs must preserve condition results and subsequent behavior; action arguments and computations require checking actual-value differences. Reusing checks must retain state information affecting future behavior. Skipping time points must miss no action or relevant state change, preserving elapsed time and all possible inputs at resumption. Timer relations must cover every feasible branch and successor. Enlarged ranges retain old and new states and are rechecked [27]. Finishing these checks establishes successor coverage and command agreement, including additional unreachable states. Terminating symbolic analyses instead require matching commands and times on every feasible path through termination.

From any allowed initial pair, each step stays within checked states with matching commands. Induction gives agreement for every finite execution sequence; time progress extends it to full timed action traces. Divergence requires replay from an allowed initial state to confirm a trace difference. Unsupported operations, execution errors, and exhausted budgets yield no equivalence verdict. This manual argument does not machine-verify the implementation.

REFERENCES

- [1] L. Shen, Q. Yang, X. Huang, Z. Ma, and Y. Zheng, “Gpiot: Tailoring small language models for iot program synthesis and development,” in *Proceedings of the 23rd ACM Conference on Embedded Networked Sensor Systems (SenSys)*. ACM, 2025, pp. 199–212.
- [2] Y. Shi, X. Liu, C. Yu, T. Yang, C. Gao, C. Liang, and Y. Shi, “Bridging the gap between natural user expression with complex automation programming in smart homes,” 2024.
- [3] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, “Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 2018, pp. 3911–3921.
- [4] M. Chen, J. Tworek, H. Jun *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021. [Online]. Available: <https://arxiv.org/abs/2107.03374>
- [5] Anonymous Authors, “VETS: Joi grammar, evaluation datasets, and verification artifact,” 2026. [Online]. Available: <https://anonymous.4open.science/r/VETS-artifact-B248>
- [6] L. Zhang, W. He, J. Martinez, N. Brackenbury, S. Lu, and B. Ur, “Autotap: Synthesizing and repairing trigger-action programs using ltl properties,” in *Proceedings of the 41st International Conference on Software Engineering (ICSE)*. IEEE/ACM, 2019, pp. 281–291.
- [7] D. T. Nguyen, C. Song, Z. Qian, S. V. Krishnamurthy, E. J. M. Colbert, and P. McDaniel, “IoTSan: Fortifying the safety of IoT systems,” in *Proceedings of the 14th International Conference on emerging Networking EXperiments and Technologies (CoNEXT)*, 2018.
- [8] Z. B. Celik, G. Tan, and P. McDaniel, “IoTGuard: Dynamic enforcement of security and safety policy in commodity IoT,” in *Network and Distributed System Security Symposium (NDSS)*, 2019. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/2019/02/ndss2019_07A-1_Celik_paper.pdf
- [9] Y. Yu and J. Liu, “Tapinspector: Safety and liveness verification of concurrent trigger-action iot systems,” *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 3773–3788, 2022.
- [10] Y. Cheng, M. Xu, Y. Zhang, K. Li, R. Wang, and L. Yang, “Autoiot: Automated iot platform using large language models,” 2024.
- [11] Y. Gao, K. Xiao, F. Li, W. Xu, J. Huang, and W. Dong, “Chatiot: Zero-code generation of trigger-action based iot programs,” *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 8, no. 3, 2024.
- [12] Y. Cheng, M. Xu, Y. Zhang, K. Li, H. Wu, Y. Zhang, S. Guo, W. Qiu, D. Yu, and X. Cheng, “Say what you mean: Natural language access control with large language models for internet of things,” 2025.
- [13] Y. Yu, Y. Xu, K. Huang, and J. Liu, “Tapfixer: Automatic detection and repair of home automation vulnerabilities based on negated-property reasoning,” in *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. USENIX Association, 2024, pp. 4945–4962. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/you-yinbo>
- [14] Q. Wang, P. Datta, W. Yang, S. Liu, A. Bates, and C. A. Gunter, “Charting the attack surface of trigger-action iot platforms,” in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2019, pp. 1439–1453.
- [15] Z. B. Celik, P. McDaniel, and G. Tan, “Soteria: Automated iot safety and security analysis,” in *Proceedings of the 2018 USENIX Annual Technical Conference (USENIX ATC)*. USENIX Association, 2018, pp. 147–158. [Online]. Available: <https://www.usenix.org/conference/atc18/presentation/celik>
- [16] C. Fu, Q. Zeng, and X. Du, “Hawatcher: Semantics-aware anomaly detection for appified smart homes,” in *Proceedings of the 30th USENIX Security Symposium (USENIX Security)*. USENIX Association, 2021, pp. 4223–4240. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/fu-chenglong>
- [17] G. Seo, J. Yang, J. Pyo, N. Kim, J. Lee, and Y. Jo, “Simuhome: A temporal- and environment-aware benchmark for smart home llm agents,” 2025.
- [18] L. Zhang, C. Zhou, M. L. Littman, B. Ur, and S. Lu, “Helping users debug trigger-action programs,” *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 6, no. 4, pp. 1–32, 2022.
- [19] L. Zheng, W.-L. Chiang, Y. Sheng *et al.*, “Judging LLM-as-a-judge with MT-Bench and chatbot arena,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html
- [20] Qwen Team, “Qwen3.5-9B model card,” 2026, accessed: Sep. 19, 2026. [Online]. Available: <https://huggingface.co/Qwen/Qwen3.5-9B>
- [21] Hyper-AI, “Qwen3.5-9B-fp8 model card,” 2026, accessed: Sep. 19, 2026. [Online]. Available: <https://huggingface.co/Hyper-AI/Qwen3.5-9B-fp8>
- [22] OpenAI, “GPT-5.4 mini model,” 2026, accessed: Sep. 19, 2026. [Online]. Available: <https://developers.openai.com/api/docs/models/gpt-5.4-mini>
- [23] Anthropic, “Introducing Claude Sonnet 5,” 2026, accessed: Sep. 19, 2026. [Online]. Available: <https://www.anthropic.com/news/claude-sonnet-5>
- [24] A. Pnueli, M. Siegel, and E. Singerman, “Translation validation,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, ser. Lecture Notes in Computer Science, vol. 1384. Springer, 1998, pp. 151–166.
- [25] E. M. Clarke, O. Grumberg, and D. A. Peled, *Model Checking*. MIT Press, 1999.
- [26] J. Bengtsson and W. Yi, “Timed automata: Semantics, algorithms and tools,” in *Lectures on Concurrency and Petri Nets*, ser. Lecture Notes in Computer Science. Springer, 2004, vol. 3098, pp. 87–124.
- [27] P. Cousot and R. Cousot, “Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fix-points,” in *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*. ACM, 1977, pp. 238–252.
- [28] L. de Moura and N. Björner, “Z3: An efficient SMT solver,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, ser. Lecture Notes in Computer Science, vol. 4963. Springer, 2008, pp. 337–340.
- [29] Google, “Example scripted automations,” Google Home Developers, accessed: Sep. 13, 2026. [Online]. Available: <https://developers.home.google.com/automations/example-scripts>
- [30] Home Assistant, “Automation triggers,” accessed: Sep. 13, 2026. [Online]. Available: <https://www.home-assistant.io/docs/automation/trigger/>
- [31] —, “Motion-activated light: Automation blueprint,” accessed: Sep. 19, 2026. [Online]. Available: https://github.com/home-assistant/core/blob/dev/homeassistant/components/automation/blueprints/motion_light.yaml
- [32] —, “Zone notification: Automation blueprint,” accessed: Sep. 19, 2026. [Online]. Available: https://github.com/home-assistant/core/blob/dev/homeassistant/components/automation/blueprints/notify_leaving_zone.yaml
- [33] B. Ur, E. McManus, M. Pak Yong Ho, and M. L. Littman, “Practical trigger-action programming in the smart home,” in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI)*, 2014, pp. 803–812.
- [34] J. Huang and M. Cakmak, “Supporting mental model accuracy in trigger-action programming,” in *Proceedings of the ACM International Joint Conference on Pervasive and Ubiquitous Computing (UbiComp)*, 2015, pp. 215–225.
- [35] W. Brackenbury, A. Deora, J. Ritchey, J. Vallee, W. He, G. Wang, M. L. Littman, and B. Ur, “How users interpret bugs in trigger-action programming,” in *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 2019.
- [36] AutoTap authors, “AutoTap: Released user study 1 data,” data - User Study 1.xlsx, Result sheet. Accessed: Sep. 13, 2026. [Online]. Available: <https://github.com/zlfben/autotap/tree/master/data>
- [37] Home Assistant Community, “How to set notifications when door is left open?” example source thread; corpus records identify individual posts. Accessed: Sep. 12, 2026. [Online]. Available: <https://community.home-assistant.io/t/how-to-set-notifications-when-door-is-left-open/520305>
- [38] macintoshee, “How to automate open door notifications?” Home Assistant Community, 2024, opening post, Oct. 18, 2024. Accessed: Sep. 19, 2026. [Online]. Available: <https://community.home-assistant.io/t/how-to-automate-open-door-notifications/783900>